Augmenting data structures

Part III Data Structures

Scope

Augmentation speeds up new queries while accounting for summary maintenance. Subtree size supports dynamic rank/select. Local-update proofs and exact maxima with closed endpoints establish why an interval tree can find overlap along one path without missing an answer.

Prerequisites

BSTs, size fields, intervals, and invariants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Support dynamic ranks with subtree sizes.
- Choose reliable auxiliary information systematically.
- Design an interval tree for overlap queries.
- 14.1 Dynamic order statistics
- 14.2 How to augment a data structure
- 14.3 Interval trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic order statistics

Chapter9 finds one kth smallest element of a static unsorted array in linear time. Repeated rank/select queries amid insertion and deletion can instead use subtree summaries in a balanced BST. An order-statistic tree augments each red-black node x with its number of real subtree nodes, including x, with NIL.size=0:

`size(x)=size(x.left)+size(x.right)+1`.

Rank is the one-based inorder position of a node identity. If duplicates occupy separate nodes, order by (key,identity) or explicitly use current inorder node order; a duplicate value has no unique rank by itself. Our examples have distinct keys.

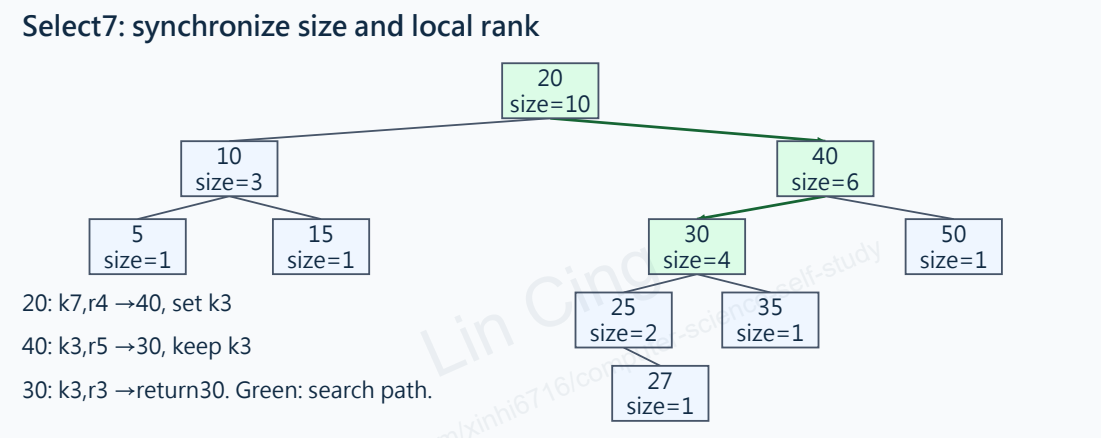
Query time is initially O(height+1). It becomes worst-case O(log(n+1)) only if the underlying tree preserves Chapter13's balancing properties. Adding size to an ordinary BST does not balance it.

Select's local-rank invariant

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

Invariant: the answer is local identity k in the current x-subtree. The left subtree has r−1 nodes and x is position r. Going left retains k; going right subtracts the entire left subtree plus x, r nodes. A valid k guarantees the chosen side is nonempty and its new k valid, so descent never incorrectly enters NIL. Each iteration descends one level and returns the correct node.

Select's local-rank invariant (continued)



At root20, left.size=3 and r=4, so select7 goes to40 with k=3. At40, left.size=4 and r=5, so it goes to30 with k still3. At30, left.size=2 and r=3, so return30. Do not compare global rank7 unchanged at every level.

Rank adds only an uncounted prefix

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

Invariant: r is original x's inorder rank inside current u's subtree. If u is a left child, the parent and right side follow x, so add nothing. If u is a right child, its parent's left subtree and parent precede x and lie outside the previously counted u-subtree. Add exactly left.size+1 without duplication. At root, r is the global rank.

For rank27, start r=1.27 is25.right, so add0+1 to get2.25 is30.left and30 is40.left, so add nothing.40 is20.right, so add3+1 to get6. Inorder5,10,15,20,25,27,30,35,40,50 confirms position6.

Update size according to the final structure

A new leaf has size1; only its ancestors gain one node. Count along the search path or recompute from its parent to root after linking. Each ancestor costs O(1), totaling O(height+1). Updating an existing duplicate record must not increment size.

Deletion cannot simply decrement ancestors of requested node z. With two children, successor y moves from its old position to z's, changing y's own subtree. A safe approach finishes Chapter12's transplants, then recomputes affected ancestors from deepest to shallowest.

Deleting20 and moving25 starts at25's old parent30: size30 changes4→3, size40 changes6→5, and new root25 becomes1+3+5=9. It cannot retain old size2. For a direct successor, start at moved y; for a leaf/one-child case, start at z's old parent.

Affected nodes lie on a constant number of ancestor paths, totaling O(height+1). Red-black repair rotations must also maintain size; the next section states when only two nodes need updates.

How to augment a data structure

Answer four questions together: what basic operations does the underlying structure support, what summary is stored, how does every mutation maintain it, and how does the new query use it to locate or prune? A formula for f alone proves neither maintenance nor query correctness.

A useful sufficient condition is that each node's f is computed in O(1) from its own key/data, unaffected by recoloring, and its children's f, with a constant NIL base case. Changed local data or child links can affect only that node and ancestors. Recompute descendants before ancestors.

The initial leaf attachment and a constant number of deletion transplants affect a constant number of O(log n) paths. For a rotation, repair the two pivot nodes first and propagate to ancestors if necessary, at most O(log n) extra work. Chapter13 bounds insertion to2 and deletion to3 rotations, so total augmentation work remains O(log n).

This general argument allows O(log n) propagation per rotation. It does not claim that every f needs only two local writes. If f depends on color, recoloring's dependency effects must also be maintained; unchanged structure alone is insufficient.

Size, maximum, and general summaries

In left-rotate(x), y=x.right rises and β=y.left moves to x. Finish pointer changes, then pull(x) before pull(y), because new y depends on new x. Reversing the order reads stale x information.

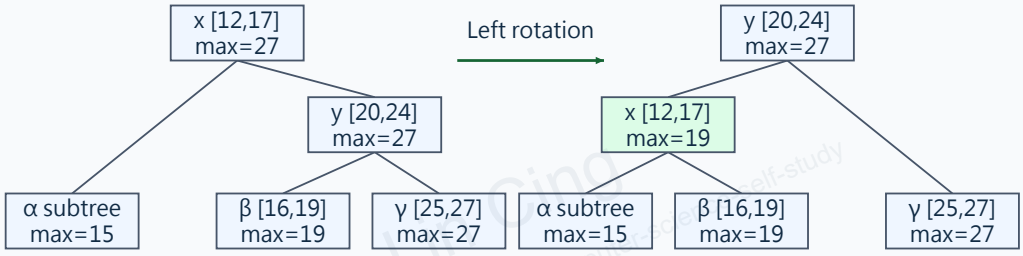
Size and maximum interval high depend only on subtree contents. New y contains exactly old x's entire subtree after rotation, so its summary equals old x's summary and other ancestors do not change. These two summaries need O(1) local recomputation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size, maximum, and general summaries (continued)

Local rotation: update the descending node first

Old x=[12,17], y=[20,24]; α.max15, β=[16,19], γ=[25,27].



Pull new x: max(17,15,19)=19; then new y: max(24,19,27)=27.

Green: descending x changes summary. Blue: other summaries stay unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size, maximum, and general summaries (continued)

Height is also child-computable, yet rotation can change the whole local subtree's height and require ancestor propagation. Explicitly storing global rank at every node requires $\Theta(n)$ writes when a new minimum raises all old ranks. Stored depth can similarly change across a large subtree after one rotation. Subtree size supports local maintenance and pathwise combination at query time, which is why it is useful.

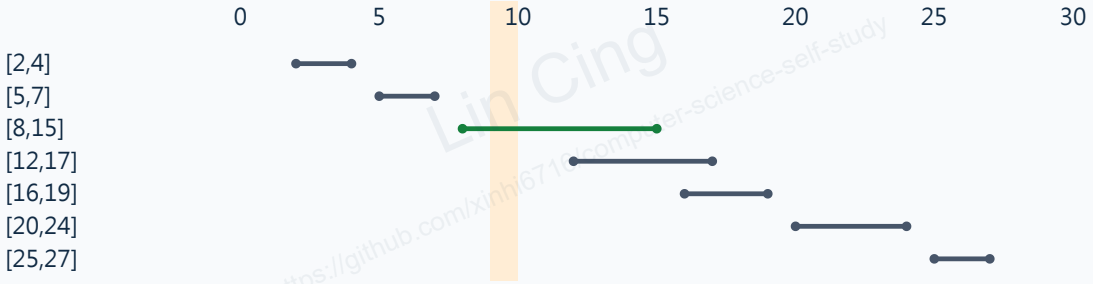
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interval trees

Intervals here are closed $[l,h]$, with $l \le h$, including singletons $[l,l]$. Intervals i,q overlap exactly when $i.low \le q.high$ and $q.low \le i.high$. Equivalently, neither lies completely left of the other: $i.high < q.low$ and $q.high < i.low$ are the two rejection cases.

Closed intervals and the same overlap query

Orange band: closed query[9,10]. Filled endpoint dots mean closed intervals.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Green[8,15]: overlaps. Gray: no overlap. [4,5] and[5,7] overlap at5.

Interval trees (continued)

Order the BST primarily by low; use identity to break equal-low ties, retaining left.low≤x.low≤right.low. Store max(x)=max(x.high,left.max,right.max), with NIL.max=−∞. This is the highest high in the **whole subtree**, not the node's high or the largest low.

Root[12,17] has high17 but max27. Left[5,7] has max15 from right descendant[8,15]. Looking only at the current interval can miss a farther-reaching descendant.

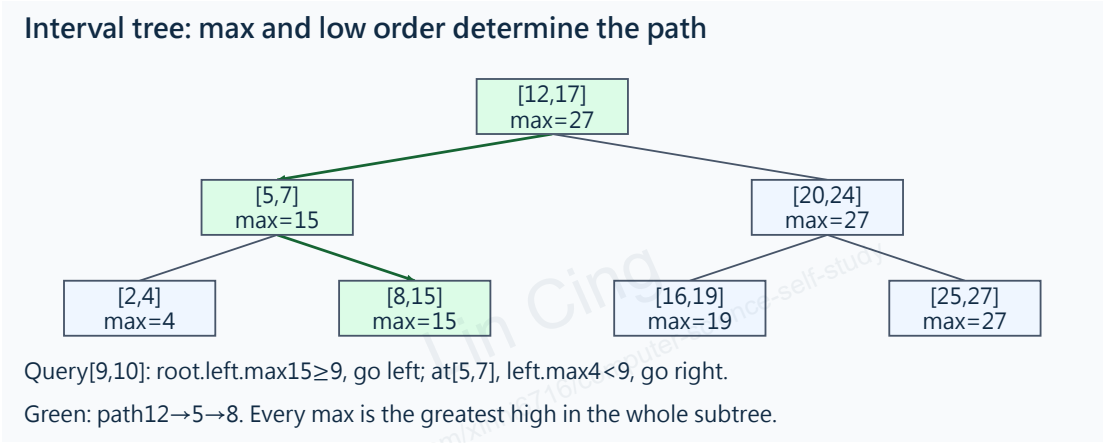
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One-path overlap search

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One-path overlap search (continued)



Query[9,10] does not overlap root[12,17], but left.max15≥9 leads to[5,7]. That also misses; left[2,4].max4<9 leads right to overlapping[8,15].

Query[18,18] goes from nonoverlapping root right because left.max15<18. At[20,24], left.max19≥18 leads to matching[16,19]. Query[28,29] follows[12,17]→[20,24]→[25,27]→NIL and fails.

Why rightward pruning is safe

Maintain: if an overlap exists anywhere in the tree, the current x-subtree contains one. This holds initially at root. Return if x overlaps. Otherwise prove that the chosen direction cannot discard all answers.

If left=NIL, it has no candidate. If left.max<q.low, every left interval j has j.high≤left.max<q.low and lies completely left of the query. Exclude the whole left side. Since x also misses, any answer must be right, preserving the invariant.

This uses a bound on every left descendant's high. The single condition x.high<q.low does not imply every left interval lies too far left. Change only the example root to[12,13] and query[14,14]: root.high13<14, yet left descendant[8,15] overlaps, so the left side cannot be pruned on that basis.

Why choosing left needs no later right search

When left.max≥q.low, choose a left interval j attaining that maximum. If left has any overlap, descending left is safe. If it has none, j.high≥q.low while j misses, so j.low>q.high; it cannot have j.high<q.low.

For every right descendant k, BST order gives `k.low≥x.low≥j.low>q.high`. The whole right side also lies too far right. With x itself nonoverlapping, this means the x-subtree had no answer. Thus either way, if an answer existed, the chosen left side retains one.

This proves the one-path algorithm. It does not assert that left.max≥q.low guarantees a left overlap. At NIL the chosen subtree is empty, so the invariant implies global absence. Each step descends once, giving O(height+1), or O(log(n+1)) with a red-black foundation.

Interval updates must allow maxima to decrease

After insertion/deletion, pull max along affected ancestors; after rotation, pull new x before new y. Merely increasing maxima on insertion is insufficient. Removing the greatest high or shortening an interval can decrease max, requiring recomputation from own high and both children.

Deleting[25,27] changes right[20,24].max27→24 and root.max27→24. This demonstrates a necessary decrease, but these two stale nodes are not the queried left-child summaries, so this deletion alone is not a concrete missed-query counterexample.

A real stale-left counterexample first changes[8,15] to[8,27], preserving low, making parent[5,7].max27. Delete[8,27]. The left subtree now has only[5,7] and[2,4], with correct max7; suppose it incorrectly stays27. Query[22,22] misses root[12,17] but follows stale left.max27≥22 to[5,7], then follows[2,4].max4<22 to empty right and returns NIL. It misses the actual overlap right[20,24]. An underestimated max can instead prune a true answer directly. Maintain **exact** max, not an arbitrary upper bound.

Interval updates must allow maxima to decrease (continued)

Closed intervals[5,7] and[4,5] overlap at5. Half-open[5,7) and[4,5) do not. Nonempty half-open intervals require l<h, strict overlap inequalities, and left.max>q.low for possible left overlap. Changing brackets alone while retaining comparisons is incorrect.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finding one, the leftmost, or all overlaps are different queries

The algorithm promises any overlapping record, not minimum low or all matches. To enumerate all, recurse left only if left.max≥q.low, test x, and recurse right only if x.low≤q.high and right.max≥q.low. Each visited node costs O(1), with worst-case O(n). The one-path O(log n) bound cannot be transferred to enumeration.

Another provable enumeration method repeats overlap-search, temporarily deletes each returned record, and records it in a list. k matches plus one final failure cost O((k+1)log(n+1)); reinsertion has the same order. This temporarily mutates the tree and must fit the caller's contract. Output alone costs Ω(k). Achieving exactly O(log n+k) requires another analysis or representation; saying “output-sensitive” is no proof.

Size and max can be stored together, each pulled in O(1), with the same maintenance order. Rank then follows BST order on (low,identity), not high order or number of overlaps, which need other summaries or algorithms.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter14.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study